

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	N. Srikanth	Reg. No.:	192411139
Section / Batch:	C8E	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I, N. Srikanth (192411139) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N. Srikanth

Q1

Binary Search - Central Region Search.

given [7, 14, 21, 28, 35, 42, 49, 56, 63]

Analysis

- * The dataset is sorted in ascending order, so Binary search is the appropriate algorithm.
- * Binary search compares the target with the middle element and discards half of the remaining elements after each comparison.

step-by-step solution:

step	low	high	mid	middle element	Result.
1	0	8	4	35	$42 > 35 \rightarrow$ search right
2	5	8	6	49	$42 < 49 \rightarrow$ search left
3	5	5	5	42	element found.

Sequence of middle elements examined:

35 $\rightarrow$ 49 $\rightarrow$ 42

Search Space Reduction:

- * $[7, 14, 21, 28, 35, 42, 49, 56, 63]$
- * $[42, 49, 56, 63]$
- * $[42]$

Number of Comparisons: 3

Time Complexity:

- * Best Case: $O(1)$
- * Average Case: $O(\log n)$
- * Worst Case: $O(\log n)$

Connectness verification:

* The element 42 is found at index 5

* The algorithm correctly identifies the target after the Comparisons.

Performance Analysis: Binary search is highly efficient because it halves the search space after every comparison. Compared to linear search, which may require up to 9 comparisons for this dataset.

Q2 Searching techniques - Comparative Analysis.

1. Problem interpretation.

The objective is to compare Linear search and Binary search by analyzing how they search for an element, the number of comparisons required, their time complexities, and when each method is most suitable.

2. Working of the Algorithms

Linear search

* checks each element one by one from the beginning until the target is found or the list ends.

Binary search

* Compares the target with the middle element.
* works only on sorted data.

3. Applying both methods

Example Target: 42

Unsorted Data: [85, 7, 49, 21, 42, 63, 14, 56, 28]

Linear search.

* Comparisons: 85 $\rightarrow$ 7 $\rightarrow$ 49 $\rightarrow$ 21 $\rightarrow$ 42

Total Comparisons = 5

Sorted Data: [7, 14, 21, 28, 35, 42, 49, 56, 63]

Binary search.

* Middle elements examined: 35 $\rightarrow$ 49 $\rightarrow$ 42.

4. Time Complexity.

Case	Linear search	Binary search
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$

5. Efficiency Comparison

Feature	Linear search	Binary search
Data Requirement	sorted or unsorted	sorted only.
Search speed	Slower	Faster
Comparisons	up to n	about $\log_2 n$
Suitable for	small or unsorted datasets	Large sorted datasets.

6. When each method is preferable.

- * use Linear search when:
 - * Data is unsorted.
 - * Dataset is small.
 - * sorting is not practical
- * use Binary search:
 - Data is already sorted.
 - Dataset is large
 - fast searching is required.